

Tensor Data Interchange: A State-of-the-Art Review

A survey of the formats, protocols, libraries, and serving systems that move, store, or consume tensor data in AI/ML inference pipelines – and of the capabilities none of them provide.

September 2026 · Also available as [PDF](prior-art.pdf)

Contents

Abstract	2
1. Introduction	2
1.1 Motivation	2
1.2 Scope	3
1.3 Contributions	3
2. Terminology: Two Roles	3
3. Background	3
3.1 No single layout is optimal	3
3.2 Zero-copy is not a micro-optimization at inference scale	4
3.3 Disaggregated inference makes transfer the dominant cost	4
4. Data Interchange Solutions	4
4.1 In-Process ABI	4
4.2 IPC and Streaming	5
4.3 File-Based	6
4.4 RDMA and Transport	7
4.5 Comparison	8
5. Compute Frameworks and Libraries	9
5.1 General-Purpose ML	9
5.2 High-Performance Linear Algebra	11
5.3 Compilers and Runtimes	11
5.4 Specialized Hardware	12
5.5 Serving Systems	12
5.6 Comparison	13
6. Case Study: KV Cache Transfer in Disaggregated Inference	14
6.1 DistServe	14
6.2 Mooncake	14
6.3 vLLM's connector interface	15
6.4 NVIDIA Dynamo and TensorRT-LLM	15
6.5 llm-d	16
6.6 LMCACHE	16
6.7 Synthesis: the metadata gap	16
6.8 What the case study demands	17
7. Region-Heterogeneous Tensor Structures	18
7.1 Comparison	18
7.2 Findings	19
8. Gap Analysis: Required Capabilities	19
8.1 Zero-copy sharing with stated alignment	20
8.2 Streaming with self-delimiting framing	20
8.3 Self-description	20

8.4 Layout negotiation	20
8.5 Device and memory-placement description	20
8.6 First-class quantization metadata	21
8.7 Language-agnostic ABI	21
8.8 Summary	21
9. Conclusion and Open Questions	21
9.1 Conclusion	21
9.2 Open questions	22
Appendix A — Glossary	22
Appendix B — Representative Tensor Shapes	23
Appendix C — Memory Layouts in Production	24
References	25

Abstract

Machine-learning systems move enormous quantities of tensor data between processes, machines, and storage tiers, yet no widely adopted mechanism carries a complete description of a tensor alongside its bytes. This review surveys the field in two parts. The first covers **data interchange solutions** — in-process ABIs, IPC and streaming protocols, file containers, and RDMA transports — and finds each expressive in one dimension and silent in the others: DLPack models strides but not tiling or quantization; Apache Arrow contributes an exemplary buffer and IPC discipline built on a tabular data model; GGUF specifies quantization well but only for a single runtime and only on disk; NIXL, NCCL, and UCX move accelerator memory at line rate while transferring anonymous byte ranges. The second part covers **compute frameworks and libraries**, which are the clients of the first group, and shows how each absorbs the shortfall as private convention, an extra copy, or an out-of-band side channel. The cost is sharpest in disaggregated large-language-model inference, where the key-value cache crosses the network for every request: every production system surveyed here fixes shape, data type, layout, and quantization out of band and ships opaque blocks with identifiers, which forces hand-written layout conversion whenever two endpoints disagree. From this evidence the review derives seven capabilities a general tensor interchange format must provide — zero-copy sharing with stated alignment, self-delimiting streaming, self-description, layout negotiation, device and memory-placement description, first-class quantization metadata, and a language-agnostic ABI — and closes with the open questions that designing such a format raises. Those seven capabilities constitute the design brief for **Hurray**, a tensor interchange format outlined in broad terms in § 8 and § 9. This document states what such a format must do, not how any particular realization does it.

1. Introduction

1.1 Motivation

A modern inference deployment is a supply chain. Weights are trained in one framework, quantized by a second, distributed in a third’s file format, loaded by a fourth runtime, and served by a fifth system that moves activations and cached attention state between machines for every request. At each handoff the data is the same — a multi-dimensional array of numbers — but the description of that data is reconstructed from scratch, usually by convention rather than by protocol.

That reconstruction is not free. It costs copies at gigabyte scale, it costs bespoke adapter code at every pair of endpoints, and it costs the ability to store a tensor and read it back with anything other than the exact software that wrote it. The question this review asks is narrow and practical: **what is missing from existing solutions, such that this cost is paid over and over?**

1.2 Scope

The survey covers solutions that carry tensor data in AI/ML inference pipelines, together with the scientific-computing formats whose array modelling is directly instructive. It excludes model-graph interchange, training-orchestration frameworks, and general-purpose serialization, none of which addresses the question above.

Each system is examined along a fixed set of dimensions: its type, its primary use case, its layout model, its quantization support, its interchange mechanism, its transport, and its adoption.

1.3 Contributions

1. A classification of the field into **data interchange solutions** and **compute frameworks and libraries**, with the dependency between them made explicit (§ 2).
 2. A structured survey of each group — the first by primary use case, the second by domain — with comparison tables (§ 4, § 5).
 3. An analysis of disaggregated LLM inference as the workload that exposes the missing capability most acutely, with evidence from six production and research systems (§ 6).
 4. A survey of region-heterogeneous array structures, a class no mainstream tensor interchange format models (§ 7).
 5. A derivation of seven required capabilities for a general tensor interchange format, each traced to the evidence that demands it (§ 8).
 6. A statement of the open questions such a format must resolve (§ 9).
-

2. Terminology: Two Roles

The literature routinely places formats, libraries, and frameworks on a single list. Separating two roles makes the analysis tractable.

Data interchange solutions move or store tensor data. They define a byte layout, a descriptor, a wire protocol, a file container, or a transport. They do not perform arithmetic. DLPack, Apache Arrow, Arrow Flight, SafeTensors, GGUF, Zarr, NetCDF, OPeNDAP, NIXL, NCCL, and UCX belong to this group.

Compute frameworks and libraries perform computation on tensors: kernels, graph execution, autotuning, request scheduling. PyTorch, TensorFlow, JAX, NumPy, Eigen, xtensor, PLASMA, SLATE, TVM, MLC-LLM, MLX, vLLM, and NVIDIA Dynamo belong to this group.

Compute frameworks are the clients of interchange solutions. PyTorch computes; DLPack carries a tensor into and out of it. vLLM schedules and generates tokens; NIXL moves its cached attention state. This dependency is the analytical instrument of the review: when an interchange solution cannot express something the client needs — a tiled layout, a quantization scheme, a paged cache, a device placement — the client absorbs the cost. Every such absorption is evidence of a missing capability, and § 8 collects them.

One entry sits on the boundary and is labelled where it does most of its work: NumPy is an array library, and therefore a compute client, but its stride model is a reference point for interchange design and is discussed as such (§ 5.1.1).

3. Background

3.1 No single layout is optimal

Matrix multiplication, the dominant operation in machine-learning workloads, is sensitive to memory layout: the same numbers stored differently execute orders of magnitude apart. Fast kernels tile the computation so that working sets fit the cache hierarchy, repack operands into contiguous panels to

remove stride penalties inside vector and matrix-unit inner loops, and adopt hardware-specific micro-layouts at the innermost level. Which layout wins depends on the operation, the hardware, and which level of the memory hierarchy saturates first.

The consequence for interchange is decisive. A format cannot mandate one layout without imposing a conversion on every producer or consumer that prefers another. It must instead **describe** whichever layout the producer already holds, precisely enough that the consumer knows what conversion, if any, it owes.

3.2 Zero-copy is not a micro-optimization at inference scale

A single weight matrix in a 70-billion-parameter model occupies hundreds of megabytes; a long-context attention cache runs to gigabytes per request (Appendix B). Copying such a buffer to satisfy an alignment rule, or to reshape it into the consumer’s expected form, consumes memory bandwidth that the computation itself needs, and doubles peak residency at exactly the moment accelerator memory is scarcest.

Zero-copy sharing is therefore a protocol requirement rather than an optimization. It depends on producer and consumer agreeing in advance on alignment, ownership, and lifetime, and no amount of implementation care recovers it after the fact.

3.3 Disaggregated inference makes transfer the dominant cost

Autoregressive generation has two phases with opposite hardware profiles. **Prefill** processes the entire prompt at once and is compute-bound; it fills the key-value (KV) cache, the stored attention keys and values that let subsequent steps avoid recomputing attention over the whole prompt. **Decode** emits one token at a time, is memory-bandwidth-bound, and reads and extends that cache. Running both phases on one accelerator couples two latency targets — time-to-first-token and time-per-output-token — that want different hardware and different batching.

Production systems therefore **disaggregate**: prefill and decode run on separate accelerators or nodes [26], [27]. The KV cache produced by prefill, of logical shape [layers, 2, heads, seq_len, head_dim] (the 2 being keys and values), must then reach the decode worker for every request. At long context lengths that is gigabytes per request, and the transfer, not the arithmetic, becomes the latency budget.

Two properties of that transfer matter for interchange. First, its size makes any copy expensive. Second, the cache is almost always stored **paged** [23]: a flat pool of fixed-size blocks plus a per-sequence block table mapping logical positions to physical blocks, so a transfer moves a list of non-contiguous blocks rather than one contiguous tensor. An interchange layer that cannot describe a paged, quantized, per-layer-organized tensor forces the two endpoints to agree out of band — which, as § 6 documents, is what every production system does.

4. Data Interchange Solutions

Grouped by primary use case: in-process ABI, IPC and streaming, file-based, and RDMA/transport. Each entry closes with a three-part takeaway — strengths (✓), limitations (✗), and the implication for a general interchange format (→).

4.1 In-Process ABI

4.1.1 DLPack

Type: in-process ABI · *Use case*: zero-copy tensor sharing between frameworks in one process · *Layout model*: strided, dense · *Quantization*: none · *Interchange*: pointer passing · *Adoption*: very high [1]

DLPack is a minimal open standard, originating in MXNet and later adopted across the Python array ecosystem, for sharing tensor memory between frameworks without copying. A managed tensor

structure carries a data pointer, shape, strides, device identifier, and data type, together with a destructor callback that transfers lifetime responsibility to the consumer. PyTorch, TensorFlow, JAX, CuPy, NumPy, TVM, and MLX all implement it.

Its layout model is exactly one mechanism: a step size per dimension. That expresses row-major and column-major order, transposes, and non-contiguous slices. It cannot express tiled or blocked layouts, space-filling-curve orders, panel-packed formats, sparse structures, or sub-byte quantization blocks. There is no field in which to place a scale factor, a zero point, or a block size, so quantization parameters cannot accompany a quantized tensor across the boundary.

- ✓ The closest thing the field has to a universal zero-copy tensor ABI; adoption is near-total and the lifetime-transfer design is sound
- ✗ Strides only; no quantization vocabulary; in-process only, with no IPC, streaming, or file representation
- → The handoff shape is right and worth preserving; the descriptor it carries is where the vocabulary must grow

4.2 IPC and Streaming

4.2.1 Apache Arrow

Type: IPC and streaming, columnar · *Use case:* language-agnostic in-memory analytics · *Layout model:* row- or column-major, tabular · *Quantization:* none in core · *Interchange:* IPC, shared memory, ABI-stable C data interface · *Adoption:* very high [2]

Arrow is a language-agnostic columnar memory format with an accompanying IPC protocol. A record batch is a table of typed, named columns, each backed by flat buffers with a specified minimum alignment of 64 bytes, and the IPC format supports zero-copy reads by memory mapping. Two aspects are directly instructive for tensor interchange: alignment is part of the specification rather than a convention, and the C data interface demonstrates that a stable, language-agnostic ABI for buffer handoff is achievable and adoptable.

Tensors enter Arrow through a fixed-shape tensor extension type, which embeds tensors as elements of a column and supports row-major and column-major order only. A consumer needing any other arrangement must repack, forfeiting the zero-copy property that motivated the format.

- ✓ The reference buffer model and IPC framing; alignment specified, not assumed; a proven language-agnostic ABI
- ✗ Fundamentally tabular — tensors are an extension of the data model, not the data model
- ✗ No tiled, blocked, or packed layouts; no standardized quantization metadata
- → The buffer discipline and IPC design transfer directly; the data model does not

4.2.2 Apache Arrow Flight

Type: IPC and streaming RPC · *Use case:* high-throughput record-batch transfer over a network · *Layout model:* inherited from Arrow · *Quantization:* none · *Interchange:* gRPC over HTTP/2 · *Adoption:* medium [3]

Flight layers an RPC framework over Arrow IPC: a server-streaming call, a client-streaming call, a bidirectional call, and metadata calls that describe what is available. Each message pairs an IPC header with a raw body. Its structural choice is the one worth studying — **the descriptor precedes the data, messages are typed, and exchange is bidirectional** — because that is precisely the shape a streaming tensor protocol needs.

The transport choice is where it fails for tensor workloads. gRPC framing requires at least one CPU copy per message and does not preserve buffer alignment, so receivers copy again before handing memory

to an accelerator or a linear-algebra kernel. Reported throughput of roughly 2–3 GB/s on a fast local network reflects those copies. At the buffer sizes of § 3.2, the copies are the entire cost.

- ✓ The right streaming RPC shape: typed messages, descriptor before data, bidirectional exchange
- ✗ gRPC framing forecloses zero-copy and destroys alignment for large buffers
- ✗ No layout negotiation, no quantization metadata, no device-memory support, no RDMA path
- → Adopt the protocol shape; reject the assumption that control plane and data plane must share a transport

4.2.3 OPeNDAP

Type: network request–response · *Use case:* remote sub-setting of scientific array data · *Layout model:* dense, row-major · *Quantization:* none · *Interchange:* HTTP with a constraint expression language · *Adoption:* high in Earth sciences [4]

OPeNDAP combines a data model derived from NetCDF, a constraint language for server-side projection and sub-setting, and an HTTP transport. A client requests a named variable restricted to an index range; the server computes the slice and streams binary data with metadata. It is the clearest existing demonstration that clients want a protocol that understands array structure — names, shapes, slices — rather than one that moves opaque bytes and leaves interpretation to convention.

- ✓ Establishes demand for structure-aware transfer, and for server-side sub-setting as a first-class protocol operation
- ✗ HTTP request–response only: no zero-copy, no device memory, no quantization, no in-process path
- → The sub-setting concept generalizes to shard and slice requests over faster transports

4.3 File-Based

4.3.1 SafeTensors

Type: file-based · *Use case:* safe distribution of model weights · *Layout model:* row-major only · *Quantization:* none · *Interchange:* file, memory-mappable · *Adoption:* high [5]

A small JSON header listing data type, shape, and byte offsets, followed by raw tensor bytes. Individual tensors can be read without deserializing the file, and loading cannot execute code — a deliberate response to the security properties of pickle-based checkpoints. It has become the default distribution format for open-weight models.

- ✓ Safe, memory-mappable, and minimal; proves that a trivially simple header plus raw buffers suffices for the distribution use case
- ✗ Row-major only; no strides, tiling, or quantization descriptors
- ✗ File-only: no IPC, no streaming, no language-agnostic ABI
- → The header/payload split is right; the vocabulary in the header is too small for runtime interchange

4.3.2 GGUF

Type: file-based · *Use case:* self-contained local inference artifacts · *Layout model:* row-major, with packed block-quantized sequences · *Quantization:* rich but informally specified · *Interchange:* file, memory-mappable · *Adoption:* high [6]

A single binary file carrying weights, tokenizer data, and hyperparameters behind an extensible key-value metadata header. Quantized tensors are stored as packed byte sequences with interleaved scale factors, in a family of schemes covering a wide range of bit widths and block structures. GGUF is the strongest working demonstration in the field that quantization metadata belongs inside the format rather than beside it: a consumer opens the file and knows how to dequantize, with no external configuration.

The limitation is that the schemes are defined by their reference implementation rather than by a portable specification, so interoperability rests on reading that code.

- ✓ Demonstrates that quantization metadata in the container is both feasible and valuable
- ✓ Best-in-class for single-user local inference
- ✗ Schemes are implementation-defined; interoperability proceeds by imitation
- ✗ File-only, effectively single-consumer; no multi-process runtime interchange
- → The quantization vocabulary must be specified normatively, and must survive transport, not only storage

4.3.3 Zarr

Type: file and object storage · *Use case*: chunked, compressed, cloud-native N-dimensional arrays · *Layout model*: chunk grid plus C or F order within chunks · *Quantization*: none natively · *Interchange*: file or object store with JSON metadata · *Adoption*: medium-high in scientific computing [7]

Arrays are divided into fixed-size chunks stored as independently compressed blobs on a filesystem, in an archive, or in an object store. The chunk grid is the closest thing in mainstream storage formats to a tiled layout description, and a recent extension adds rectilinear variable-size chunking [38]. Compression is central to the design, which places Zarr structurally at odds with zero-copy runtime access: the stored bytes are not the bytes the consumer computes on.

- ✓ The reference model for chunked, cloud-native array storage; chunk grids are a useful precedent for describing tiling
- ✗ Compression-first; no IPC protocol and no shared-memory semantics
- → Complementary rather than competing: storage and runtime interchange are different problems, and pairing a compressed storage format with an uncompressed runtime format is a proven pattern in adjacent fields

4.3.4 NetCDF

Type: file-based · *Use case*: array-oriented scientific data · *Layout model*: dense, row-major · *Quantization*: none · *Interchange*: file · *Adoption*: very high in Earth sciences [8]

NetCDF stores N-dimensional variables with named dimensions, attributes, and a small set of primitive types, in both a binary form and a text representation. Its lasting contribution is conventional rather than structural: named dimensions and rich attribute metadata make a stored array self-explanatory to a human and to generic tooling. Scaling conventions exist for compact storage but are attribute conventions rather than first-class metadata, and are therefore interpreted differently by different readers — a small, instructive preview of what happens when numeric scaling is left to convention.

- ✓ Named dimensions and attribute conventions make stored arrays self-explanatory
- ✗ Dense row-major only; no strides, tiling, or sparsity
- ✗ No in-process ABI, no IPC, no zero-copy semantics
- → Self-description is worth adopting; leaving numeric scaling to convention is not

4.4 RDMA and Transport

4.4.1 NIXL

Type: RDMA transport library · *Use case*: KV cache migration between inference workers · *Layout model*: none — registered byte ranges · *Quantization*: none · *Interchange*: RDMA with accelerator-direct paths · *Adoption*: emerging, and rapid [9]

A transfer library built specifically for high-throughput tensor movement in LLM inference. The sender registers an accelerator memory region with the network interface controller using accelerator-direct RDMA; the receiver pre-allocates an aligned buffer and shares its remote key and address; the sender issues a remote write or read, and the controller moves data directly between accelerator memories

across the network, with no host staging and no CPU involvement. Multiple backends cover network fabrics, direct storage, and storage over fabrics.

What it deliberately does not define is any tensor description: no shape, no data type, no layout, no quantization, and no negotiation. Both endpoints must already agree on the format, by means outside the library.

- ✓ The best available data plane — genuine accelerator-to-accelerator zero-copy at line rate
- ✗ Transfers anonymous byte ranges; no tensor vocabulary of any kind
- ✗ Format agreement is assumed out of band, separately for every pair of endpoints
- → A metadata plane composes with this rather than replacing it: describe the tensor, then let the network controller move it

4.4.2 NCCL and accelerator-direct RDMA

Type: RDMA transport, collectives · *Use case:* accelerator-to-accelerator communication · *Layout model:* none — flat buffers · *Quantization:* none · *Interchange:* RDMA and high-speed local interconnect · *Adoption:* very high [10], [46]

The standard collective communications library for accelerator clusters, implementing all-reduce, all-gather, reduce-scatter, broadcast, and point-to-point send and receive. When endpoints sit on different nodes connected by a suitable fabric, it uses accelerator-direct RDMA. Tensor-parallel weight splits and pipeline-parallel activation handoffs both ride its point-to-point path, increasingly in inference and not only in training.

Its tensor model is a pointer, an element count, and a data type. All layout semantics belong to the caller.

- ✓ The default, hardware-tuned transport primitive for accelerator clusters
- ✗ No tensor model, no streaming framing, no negotiation, no quantization
- → Confirms that the transport layer is solved and the description layer is not

4.4.3 UCX

Type: RDMA transport abstraction · *Use case:* transport-agnostic remote memory operations · *Layout model:* none · *Quantization:* none · *Interchange:* network fabrics, TCP, shared memory, accelerator IPC · *Adoption:* high as infrastructure [11]

A unified API for remote put and get, atomics, and stream operations that dispatches to the best available transport per connection and falls back to TCP when no fabric is present. It is not a user-facing protocol but the substrate other systems build on, including MPI implementations, collective libraries, transfer libraries, and distributed object stores.

- ✓ The practical implementation target for any RDMA-based data plane; transport selection and fallback are a solved problem
- ✗ Not a format: nothing in it describes a tensor
- → A tensor protocol should sit above such a layer, owning the registration handshake, descriptor exchange, and session state, and delegating the transfer itself

4.5 Comparison

Solution	Type	Layout model	Quantization	Interchange method	RDMA	Adoption
DLPack [1]	In-process ABI	Strided dense	✗	Pointer passing	✗	Very high
Apache Arrow [2]	IPC / streaming	Row/column-major, tabular	✗	IPC, shared memory, C data interface	✗	Very high

Solution	Type	Layout model	Quantization	Interchange method	RDMA	Adoption
Arrow Flight [3]	IPC / streaming RPC	Row/column-major (inherited)	✗	gRPC over HTTP/2	✗	Medium
OPeNDAP [4]	Network request-response	Dense row-major	✗	HTTP with constraint expressions	✗	Medium
SafeTensors [5]	File-based	Row-major	✗	File, memory-mapped	✗	High
GGUF [6]	File-based	Row-major + packed quantization blocks	✓ informal	File, memory-mapped	✗	High
Zarr [7]	File / object store	Chunk grid + C/F order	✗ (codecs only)	File / object store	✗	Medium
NetCDF [8]	File-based	Dense row-major	✗	File	✗	High
NIXL [9]	RDMA transport	None (byte ranges)	✗	RDMA, accelerator-direct	✓	Emerging
NCCL [10]	RDMA transport, collectives	None (flat buffers)	✗	RDMA, local interconnect	✓	Very high
UCX [11]	RDMA abstraction	None	✗	Fabrics, TCP, shared memory, accelerator IPC	✓	High (infrastructure)
Required (§ 8)	ABI + streaming + file + RDMA	Strided, tiled, space-filling, sparse, paged, composite, extensible	✓ first-class	Pointer handoff, stream, file, RDMA	✓	—

Table 1: Data interchange solutions

The final row states the capability set derived in § 8. It describes a requirement, not an existing artifact.

5. Compute Frameworks and Libraries

These are the clients. Each entry records which interchange solutions the system already speaks, because that is where a new format would attach, and what it does internally that the interchange layer cannot express.

5.1 General-Purpose ML

5.1.1 NumPy

Domain: general-purpose array computing · *Interchange spoken:* DLPack, the Python buffer protocol, an array interface, its own container format · *Internal layout:* arbitrary strides, dense [12]

The de facto array standard in Python. An array object carries a pointer, shape, strides, data type, and flags; transpose, slice, and broadcast are all zero-copy view operations. The stride model it popularized is the one DLPack inherited, and its data-type vocabulary is the lingua franca that lets tensors cross

between Python libraries without translation. It models no tiled, packed, or sparse layouts, and no quantization.

- ✓ The reference stride model, and the data-type vocabulary the ecosystem shares
- ✗ Language-coupled; no interchange path of its own beyond the Python ecosystem
- → A new format should match this data-type vocabulary exactly, so that the most common on-ramp requires no translation

5.1.2 PyTorch

Domain: general-purpose ML · *Primary use case:* training and inference; the substrate of most serving systems · *Interchange spoken:* DLPack, SafeTensors, collective libraries, RDMA transfer libraries by way of serving frameworks · *Internal layout:* strided dense, plus named memory formats such as channels-last [13]

Quantization support is real but fragmented: an older quantized-tensor mechanism with integer data types, and newer packed low-bit and 8-bit floating-point representations from companion libraries and third-party kernels, each with its own convention for where scales and zero points live. None of it survives a DLPack handoff, because DLPack has nowhere to put it. A quantized tensor is therefore portable only as a pair of objects — the packed data and a framework-specific parameter structure — travelling by different routes.

- ✓ Adopts DLPack as its interchange path; ecosystem gravity makes it the on-ramp that matters most
- ✗ No multi-layout descriptor and no native RDMA tensor protocol; both are added per project
- ✗ Quantization parameters travel beside the tensor, never with it
- → The needed capability is visible precisely here: the same handoff, carrying layout and quantization

5.1.3 TensorFlow

Domain: general-purpose ML · *Primary use case:* training and production serving · *Interchange spoken:* DLPack, its own saved-model container, collective libraries · *Internal layout:* strided dense, with physical layouts chosen by the compiler [14]

Quantization is expressed principally in the mobile and edge runtime, as affine per-tensor and per-axis schemes attached to the model artifact rather than to a transferable tensor. Physical layout decisions belong to the compiler and are not observable at the interchange boundary.

- ✓ Speaks DLPack; mature serving story
- ✗ Layout decisions are internal and inexpressible on the wire
- ✗ Quantization is a property of the saved model, not of an interchangeable tensor
- → A quantized tensor needs an identity independent of the model artifact that produced it

5.1.4 JAX

Domain: general-purpose ML · *Primary use case:* research and large-scale accelerator workloads · *Interchange spoken:* DLPack, checkpoint libraries, collective libraries · *Internal layout:* compiler-managed, including dimension-order permutation and tiling; explicitly sharded across devices [15]

JAX makes sharding a first-class program concept: an array is annotated with how it is distributed across a device mesh, and the compiler chooses physical layouts, including tiled ones, during compilation. Neither the sharding nor the physical layout is expressible in a DLPack handoff, so a cross-framework transfer degrades to a dense, single-device view of data that was neither.

- ✓ The strongest first-class sharding model in the group
- ✗ Sharding and physical layout are invisible at the interchange boundary
- → Shard description — a tensor's position and extent within a larger logical tensor — is a required part of the vocabulary, not an optional extension

5.2 High-Performance Linear Algebra

5.2.1 Eigen

Domain: HPC dense linear algebra · *Primary use case:* in-process C++ matrix computation · *Interchange spoken:* none, beyond mapping over caller-owned memory · *Internal layout:* row- or column-major fixed at compile time; arbitrary strides when mapping [16]

Expression templates provide lazy evaluation without temporaries. Storage order is a compile-time template parameter, so it is fixed by the program rather than negotiated at run time. The ability to map over externally owned memory is the relevant precedent: adopting a foreign buffer is a solved and widely used pattern in numerical C++, provided its layout and alignment are known.

- ✓ Demonstrates that foreign-memory adoption is routine, given a stated layout and alignment
- ✗ No serialization, no IPC, no descriptor of any kind
- → A stable, language-agnostic ABI is what such a library would map from

5.2.2 xtensor

Domain: HPC and general array computing · *Primary use case:* NumPy-like C++ arrays with Python bindings · *Interchange spoken:* the Python buffer protocol, external buffer adaptors · *Internal layout:* strided dense [17]

Deliberately more interoperable than Eigen — lazy evaluation plus adaptors over externally owned memory — but with no formal protocol of its own; interoperability stops at the Python boundary.

- ✓ External buffer adaptors make it an easy consumer of a foreign descriptor
- ✗ No cross-language or on-the-wire path
- → The consumer side is ready; the protocol is what is missing

5.2.3 PLASMA and SLATE

Domain: HPC dense linear algebra · *Primary use case:* multicore and distributed-memory solvers · *Interchange spoken:* none · *Internal layout:* PLASMA stores matrices as independently allocated tiles; SLATE supports column-major, tiled, and band layouts with mixed precision [18], [19]

PLASMA stores a matrix as individually allocated tiles so that tasks can execute asynchronously across cores. SLATE is the distributed-memory redesign, adding accelerator offload and several coexisting layouts within one library. Together they are the clearest evidence from outside machine learning that **tiled layouts are mandatory for high-performance dense linear algebra, not exotic** — and that a serious numerical library supports several layouts simultaneously rather than choosing one.

- ✓ Direct evidence that a layout vocabulary needs tiling, and needs more than one entry
- ✓ SLATE’s coexisting layouts are a model for a layout taxonomy
- ✗ Neither defines an interchange format; layout descriptions are private to the library
- → Those same tile parameters must be describable across a process boundary

5.3 Compilers and Runtimes

5.3.1 TVM and MLC-LLM

Domain: ML compiler and runtime · *Primary use case:* compiling and autotuning models for heterogeneous hardware · *Interchange spoken:* DLPack natively; compiled module plus parameter blob artifacts · *Internal layout:* aggressively rewritten during compilation — tiling, channel-blocked convolution layouts, matrix-unit fragment layouts [20], [21]

TVM ingests models, lowers them through a graph intermediate representation to a tensor intermediate representation, autotunes kernels, and emits code for CPUs, accelerators, and microcontrollers; MLC-LLM builds on it to run language models across phones, laptops, and browsers. TVM originates from the

same lineage as DLPack, and its runtime array type is DLPack-native — which independently validates the in-process ABI shape from the compiler side.

Its layout rewriting is the instructive part. The compiler converts tensors into hardware-preferred packed or tiled forms because that is where the performance is. If a producer has already done that work, an interchange format unable to describe the result forces it to be undone and redone. Quantization here is a compilation pass rather than a portable representation, and parameters ship as ad-hoc bundles.

- ✓ Confirms the in-process ABI choice from the compiler side
- ✓ Names precisely the packed and tiled forms a producer might hand over pre-optimized
- ✗ No framework-agnostic, quantization-aware container for its parameter blobs
- → Two roles for a general format follow: the zero-copy handoff at run time, and a layout- and quantization-aware container for compiled weights

5.4 Specialized Hardware

5.4.1 MLX

Domain: specialized hardware — unified-memory systems · *Primary use case:* on-device ML research and inference · *Interchange spoken:* DLPack, the Python buffer protocol; loads several existing file formats · *Internal layout:* strided dense; quantized weights packed into unsigned 32-bit words [22]

Arrays live in memory that both the host processor and the accelerator address directly, with no copies and no explicit device placement, and computation is lazy so that graphs optimize before execution. Quantization is an *operation* rather than a storage data type: quantizing produces a packed integer weight tensor plus separate scale and bias arrays, with configurable group sizes and bit widths, alongside block floating-point modes with a shared exponent per group [45]. MLX defines no file format of its own, loading and saving through existing ones — a deliberate choice, and the correct posture for a compute framework.

Two findings bear on interchange design. First, **device affinity may belong to access rather than to storage**: under unified memory the host/device buffer duality disappears, so what carries a device identity is the kernel that reads a buffer, not the buffer itself. Any device field in a descriptor must accommodate that rather than assume every buffer belongs to exactly one device. Second, its documented sub-byte packing order — element zero in the lowest-order bits of the first word — is a concrete compatibility constraint: sub-byte packing must be specified bit-exactly, because two reasonable conventions exist and they are not interconvertible without a pass over the data.

- ✓ Multi-mode quantization packing with variable group sizes is a useful cross-check on any proposed quantization vocabulary
- ✓ Reuses existing interchange formats rather than inventing one — the intended posture for a compute framework
- ✗ In-process only; no IPC, streaming, or cross-language ABI
- → Device description must be expressive enough for unified memory, and sub-byte packing must be bit-exact in the specification

5.5 Serving Systems

5.5.1 vLLM

Domain: serving · *Primary use case:* high-throughput LLM inference · *Interchange spoken:* RDMA transfer libraries, external cache layers, collective libraries, existing weight file formats · *Internal layout:* paged KV cache with a per-sequence block table; strided dense activations [23]

vLLM introduced paged attention, which stores the KV cache as fixed-size blocks in a pool with a per-sequence block table, eliminating fragmentation and making prefix sharing a matter of pointing two sequences at the same block. It has since generalized cache transfer into a pluggable connector

interface, examined in § 6.3 — the single most important integration surface in this survey, and the clearest illustration of the missing capability, because what flows through that interface is an opaque block buffer plus an identifier.

- ✓ The abstraction is already right: one transport-agnostic connector boundary, many transports behind it
- ✗ Shape, data type, layout, and quantization are fixed once at startup and assumed thereafter
- ✗ Cross-engine, cross-version, or cross-quantization transfer requires a bespoke adapter
- → A self-describing descriptor at that boundary is what makes the connector general

5.5.2 NVIDIA Dynamo and TensorRT-LLM

Domain: serving · *Primary use case:* datacenter-scale disaggregated inference · *Interchange spoken:* RDMA transfer libraries, transport abstractions, MPI, collectives · *Internal layout:* paged KV cache tiered across accelerator memory, host memory, and storage [24], [25]

A serving framework with cache-aware routing and autoscaling, an inference engine backend, and a framework-agnostic block manager that tiers cached attention state across memory and storage. Its most revealing behaviour, examined in § 6.4, is layout conversion performed during transmission when the two endpoints use different parallelism strategies — code that exists because no portable description accompanies the bytes.

- ✓ The most operationally mature disaggregated stack, and framework-agnostic at the memory management layer
- ✗ Its cross-parallelism conversion module exists only because the wire carries no layout description
- → A layout-aware descriptor generalizes such a module out of existence

5.6 Comparison

Framework / Library	Domain	Primary use case	Interchange spoken	Internal layout model
NumPy [12]	General-purpose array	Python array manipulation	DLPack, buffer protocol, own container	Arbitrary strides, dense
PyTorch [13]	General-purpose ML	Training and inference substrate	DLPack, SafeTensors, collectives, RDMA via serving stacks	Strided dense + named memory formats
TensorFlow [14]	General-purpose ML	Training and production serving	DLPack, saved-model container, collectives	Strided dense; compiler-chosen physical layout
JAX [15]	General-purpose ML	Research, large-scale accelerators	DLPack, checkpoint libraries, collectives	Compiler-managed, tiled; explicitly sharded
Eigen [16]	HPC linear algebra	In-process C++ matrix computation	None (maps caller memory)	Row/column-major at compile time; strided maps
xtensor [17]	HPC / arrays	NumPy-like C++ with Python bindings	Python buffer protocol	Strided dense
PLASMA [18]	HPC linear algebra	Multicore tiled solvers	None	Tiled, independently allocated
SLATE [19]	HPC linear algebra	Distributed-memory and accelerator solvers	None	Column-major, tiled, band; mixed precision

Framework / Library	Domain	Primary use case	Interchange spoken	Internal layout model
TVM [20]	Compiler / runtime	Compile and autotune for any hardware	DLPack	Rewritten: tiled, channel-blocked, fragment layouts
MLC-LLM [21]	Compiler / runtime	LLMs on heterogeneous consumer hardware	DLPack; ad-hoc parameter bundles	Compiler-generated; grouped low-bit weights
MLX [22]	Specialized hardware	On-device ML on unified memory	DLPack, buffer protocol, several file formats	Strided dense; packed quantized words
vLLM [23]	Serving	High-throughput LLM inference	RDMA transfer libraries, cache layers, collectives	Paged KV cache + strided activations
NVIDIA Dynamo [24], [25]	Serving	Datacenter disaggregated inference	RDMA transfer libraries, transport abstractions, MPI	Paged KV cache tiered across memory and storage

Table 2: Compute frameworks and libraries

6. Case Study: KV Cache Transfer in Disaggregated Inference

Section 4.4 surveyed transport primitives; this section examines the systems built on them. Every system here is a **compute framework** that depends on a **data interchange solution** to move its KV cache. They are treated together because they are the largest real-world consumers of a tensor transfer layer, and because they expose the missing capability more clearly than any format comparison can: they all move opaque byte buffers, and every framework involved must reconstruct the meaning out of band.

Section 3.3 established what moves and why. What follows is what each system puts on the wire.

6.1 DistServe

Role: research serving system · *Interchange used:* high-speed local interconnect [26]

The system that introduced splitting prefill and decode across accelerators to optimize *goodput* — requests served within both latency targets simultaneously. It transfers the KV cache layer by layer, and its central technique is bandwidth-aware placement: co-locate a request’s prefill and decode segments so that the transfer rides a high-bandwidth intra-node interconnect, which makes transfer cost negligible relative to recomputation. Placement is selected by simulation-driven search over parallelism configurations. It reports substantial improvements in sustained request rate and in achievable latency targets against co-located baselines.

Metadata model: none beyond layer and block references. Both phases run an identical model build with an identical cache layout, assumed out of band; only numerical blocks move.

- ✓ Established the disaggregation architecture, and the finding that transfer cost dominates placement decisions
- ✗ Avoids the metadata problem by assuming homogeneous endpoints on a fast fabric
- → That assumption is exactly what production deployments cannot make

6.2 Mooncake

Role: production serving platform · *Interchange used:* its own multi-NIC RDMA transfer engine [27]

A cache-centric disaggregated platform operating at very large scale. It harvests otherwise idle host processors, memory, and storage across an accelerator cluster into a **disaggregated cache pool**,

fronted by a scheduler that maximizes throughput under latency targets while maximizing reuse of cached prefixes.

Its transfer engine is the most influential artifact: a standalone, reusable, zero-copy RDMA library that aggregates multiple network interfaces per host and selects paths from a topology matrix each server broadcasts, classifying interfaces into preferred and secondary lists per registered memory region. It prefers locality-optimal accelerator-direct paths and fails over on error, reporting aggregate bandwidths several times that of TCP on the same hardware.

Metadata model: transfers are keyed by block identifiers and offsets into registered memory regions. Shape, data type, and paged layout are engine configuration, agreed out of band.

- ✓ The most complete production demonstration that cached attention state is worth treating as poolable, transferable, durable data
- ✓ Its transfer engine is direct prior art for any RDMA data plane
- ✗ Still moves registered byte ranges rather than described tensors
- → A pooled cache with no self-description is reusable only by the software that wrote it

6.3 vLLM’s connector interface

Role: inference engine · *Interchange used:* several RDMA and cache-layer backends behind one interface [23], [28]

vLLM’s connector framework for disaggregated prefill and cache offloading is the de facto integration point for the ecosystem. It splits by role: scheduler-side methods decide which blocks to load or save and assemble per-step metadata, while worker-side methods register accelerator memory and run asynchronous block transfers. Transport is cleanly decoupled from model logic, and several implementations coexist behind the interface — RDMA transfer libraries, external multi-tier cache layers, local offloading, and combinations of these.

The RDMA implementation performs a side-channel handshake exchanging transfer-agent identity and memory descriptors, after which workers compute descriptor identifiers for block arrays. It also handles the case where the two endpoints run at **different tensor-parallelism degrees**, which requires reshuffling the mapping from logical blocks to physical locations.

Metadata model — the crux: shape, data type, layout, and quantization are established **once, out of band, at cache-registration time during startup**, then assumed constant per layer. Per transfer, only raw block buffers plus block and request identifiers cross the wire. The handshake negotiates addresses and agent identity; it never negotiates a tensor descriptor.

- ✓ The abstraction is already correct: one connector boundary, many transports
- ✗ What crosses that boundary is an opaque buffer plus an identifier
- ✗ Every connector pair therefore assumes identical engine builds at both ends
- → The clearest available statement of the missing capability, and the most natural place to supply it

6.4 NVIDIA Dynamo and TensorRT-LLM

Role: serving framework and inference engine · *Interchange used:* RDMA transfer library, transport abstraction, MPI [24], [25]

A datacenter-scale serving framework with cache-aware routing and autoscaling, an inference engine backend, an RDMA transfer library, and a framework-agnostic block manager that tiers cached state across accelerator memory, host memory, local storage, and remote storage, freeing accelerator memory while preserving cache hit rates. The block manager works across several inference engines, which makes it the most framework-neutral component in the survey.

The revealing detail is **layout conversion during transmission**. When the prefill and decode phases use different parallelism strategies, the engine converts the cache layout inside a dedicated exchange

module, described by its authors as modularly decoupled from both the cache manager and the communication libraries. The metadata accompanying a transfer carries prompt tokens, the first generated token, and connection parameters — how to connect and which request this is, never what shape or layout the data has.

- ✓ The most operationally mature stack, and framework-agnostic at the memory-management layer
- ✗ A layout conversion module had to be written by hand because no portable description exists to drive a general one
- → That module is precisely what a layout-aware descriptor generalizes

6.5 llm-d

Role: cluster-native serving framework · *Interchange used:* RDMA transfer library with a unified collective backend [29]

A Kubernetes-native distributed inference framework that treats prefill/decode disaggregation as a first-class orchestration primitive, built on an existing inference engine and a request gateway that routes with awareness of which worker already holds a given prefix. Cache transfer is accelerator-to-accelerator over RDMA, with tiered storage behind it, and a recent release unified over vendor collective libraries beneath the transfer layer. It reports substantial throughput and time-to-first-token improvements against monolithic deployments.

Metadata model: inherits the connector model of § 6.3 for cache movement, and adds cluster-level routing and placement metadata. The payload is still opaque engine blocks.

- ✓ Signals that disaggregated cache transfer is consolidating into shared infrastructure
- ✗ Adds orchestration metadata, not tensor metadata
- → Consolidation is exactly when a common descriptor becomes valuable rather than premature

6.6 LMCache

Role: cache layer for inference engines · *Interchange used:* engine connectors plus its own multi-tier store [30], [31], [32]

A cache layer that lifts cached attention state out of accelerator memory and shares it across engines and queries. The store spans accelerator memory, pinned host memory, local disk, and remote key-value backends, behind a modular connector and a control API for pinning, lookup, eviction, movement, and compression. Its distinctive contributions go beyond transport: one component compresses the cache into a compact bitstream for storage and transfer [31], and another enables reuse of *non-prefix* cached chunks by selectively recomputing the parts that cross-attend [32]. It reports large throughput improvements on multi-round question answering and document analysis.

Metadata model: keyed chunks — and once compressed, the payload is a codec-specific bitstream, so even the bytes are no longer a plain tensor buffer.

- ✓ Pushes furthest past raw-buffer transfer: compresses and reshapes cached state for reuse
- ✗ A compressed blob is meaningless without out-of-band knowledge of shape, data type, layout, and codec
- → Makes the absence of self-description most acute: the description problem grows as the representation gets cleverer

6.7 Synthesis: the metadata gap

Across every system above, one pattern holds: **the KV cache moves as opaque, engine-private bytes, and everything needed to interpret those bytes is agreed out of band.**

- The logical tensor, its data type, its paged block layout, and its quantization scheme are fixed by the model build and the engine configuration; they are not transmitted.

- What crosses the wire is block buffers plus identifiers, or connection and request parameters, or a codec-specific compressed blob.
- The handshakes negotiate addresses, agent identity, and parallelism mapping — never a portable tensor descriptor.

System	Interchange solution used	Travels <i>with</i> the bytes	Assumed out of band
DistServe [26]	Intra-node interconnect	Layer and block references	Identical model build and layout
Mooncake [27]	Multi-NIC RDMA transfer engine	Block keys and offsets	Shape, data type, paged layout
vLLM connector [23], [28]	RDMA and cache-layer backends	Raw blocks + block identifiers	Format fixed at registration time
Dynamo / TensorRT-LLM [24], [25]	RDMA library, transport abstraction, MPI	Tokens and connection parameters	Layout; parallelism mismatch handled by a bespoke module
llm-d [29]	RDMA library with unified collective backend	Blocks + routing hints	Model identity, layout
LMCache [30]	Engine connectors, multi-tier store	Compressed chunks + keys	Engine cache format and codec

Table 3: What travels with the bytes

Three consequences follow, and they are the symptoms a descriptor removes.

1. **Point-to-point coupling.** Every connector pair assumes identical builds at both ends. Cross-engine, cross-version, or cross-quantization transfer needs a bespoke adapter, because there is no neutral representation for the two sides to meet in.
2. **Hand-written layout conversion.** Mismatched parallelism forces ad-hoc reshuffling code instead of a transform driven by a description of the source and destination layouts.
3. **Opaque storage.** Cached state spilled to a pool or compressed to disk carries no standardized self-description, so only the software that wrote it can read it back — which negates much of the value of pooling it in the first place.

6.8 What the case study demands

The case study yields four concrete requirements, each traceable to a specific failure above.

- **Indirect layouts must be describable.** A paged cache is not a strided array. It is a pool of fixed-size pages, a table mapping logical positions to physical pages, and per-sequence page lists, with pages deliberately shared between sequences that have a common prefix. A format whose layout vocabulary stops at strides cannot describe the single most-transferred tensor in production inference.
- **The description must travel per transfer, not per session.** Fixing the format at registration time is what couples endpoints to identical builds. A description that accompanies each transfer is what makes a connector general.
- **Metadata plane and data plane must be separable.** The transports of § 4.4 are excellent and should be used as they are. What is missing sits above them: a description that names the tensor, exchanged on a control path, while the bytes move by the fastest available route.
- **Quantized and compressed payloads need naming too.** As § 6.6 shows, the trend is toward representations that are further from a plain buffer, not closer. A description that can name only dense arrays will describe a shrinking fraction of what actually moves.

7. Region-Heterogeneous Tensor Structures

Every layout in § 4 describes a *homogeneous* array: one element type, one layout, one optional quantization scheme across the whole index space. A separate body of prior art describes a *single logical array whose index space is partitioned into regions that differ in structure* – some dense, some sparse, some constant, each with its own backing storage and sometimes its own precision. This class is surveyed separately because no mainstream tensor interchange format models it at all, and because it recurs independently in three unrelated segments.

Two composition models appear in the literature, and they are not interchangeable.

- **Partition (exact cover, non-overlapping):** disjoint boxes tile the index space exactly, and each element belongs to exactly one region. Prior art: adaptive-mesh box layouts [33], [34], hierarchical volume tiles [35], and virtual datasets assembled from multiple sources [36] (which relax the rule to permit gaps).
- **Overlay (overlapping composition):** a base spanning the whole index space plus sparse corrections at scattered positions sharing indices with the base. Prior art: outlier-aware quantization [40], [41], and timestamped array fragments [37].

The distinction matters for format design: a primitive built for exact-cover partitioning cannot express an overlay, and forcing one into the other silently changes the semantics of overlapping indices. A format intending to cover both must therefore treat the composition rule itself as part of the description.

7.1 Comparison

Structure	Segment	Partition shape	Per-region inner layout	Per-region buffers	Per-region precision	Composition model	Maturity
Adaptive-mesh box layouts [33], [34]	HPC adaptive mesh refinement	Irregular boxes	Uniform dense	✓ independent	✗	Partition (exact cover per level)	Production
Hierarchical volume tiles [35]	VFX / graphics	Hierarchical tiles plus dense leaves	Heterogeneous (constant tile vs dense leaf)	✓ (linearized in the GPU form)	Partial (per-node value quantization)	Partition (hierarchical)	Production
Virtual datasets [36]	Scientific storage	Arbitrary rectangular selections	Heterogeneous (per source dataset)	✓ per source	Via per-source compression	Partition, permitting gaps and overlap	Standardized
Timestamped array fragments [37]	Array databases	N/A (temporal)	Dense plus sparse fragments	✓ per fragment	✗	Overlay (last writer wins)	Production
Variable-size chunk grids [38]	Scientific storage	Rectilinear variable grid	Uniform	✓ per chunk	✗ (array-level codec)	Partition (rectilinear only)	Emerging
Sparse tensor level encodings [39]	ML compilers	Per-dimension level, not per-region	Per-level type only	N/A	✗	Neither (whole-tensor encoding)	Production
Outlier-aware quantization [40], [41]	ML quantization	Scattered points, not rectangular	Dense low-bit plus sparse outliers	✓ (base plus outliers)	✓ (base vs outlier precision)	Overlay	Research → adoption
Residual precision caches [42]	ML inference	Regular split along the	Uniform (full vs	✓	✓ per region	Partition (regular,	Production

Structure	Segment	Partition shape	Per-region inner layout	Per-region buffers	Per-region precision	Composition model	Maturity
		sequence axis	reduced precision)			two regions)	
Per-expert quantization	ML inference	Regular expert blocks	Uniform	✓ per expert	✓ per expert	Partition (regular)	Research → adoption
Block-sparse attention [43]	ML inference	Regular block grid	Uniform dense plus mask	✗	✗	Partition (regular) plus mask	Production
Block-compressed textures [44]	GPU graphics	Regular block grid	Per-block mode	✗ (packed)	✓ per block	Partition (regular)	Production (hardware)

Table 4: Region-heterogeneous array structures

7.2 Findings

- **Irregular exact-cover partitioning with independent per-region buffers is mature**, not speculative: it is production-proven in adaptive mesh refinement and in volumetric graphics storage. The pointerless, GPU-friendly linearization of a heterogeneous-region tree [35] is a direct precedent for encoding such a structure as a zero-copy byte image, which is exactly what a streamable interchange format requires.
- **The closest standardized analogue is the virtual dataset** [36]: a logical N-dimensional dataset defined as per-region mappings onto heterogeneous backing storage. Notably, it chose *permissive* coverage, allowing gaps and overlap — a decision with consequences for validation that any format adopting the pattern must make deliberately.
- **Mainstream ML wants per-region precision on regular partitions**. The recurring cases — splitting a cache into recent full-precision and older quantized regions [42], assigning different bit widths per expert — are regular, two-or-few-region partitions. This demands a per-region precision mechanism more urgently than it demands irregular geometry.
- **The dominant ML heterogeneity pattern is an overlay, not a partition**. Outlier-aware quantization [40], [41] keeps a dense low-precision base and a scattered high-precision correction over shared indices. It cannot be expressed as a non-overlapping partition, which is why the composition rule must be explicit in the description rather than implied by the layout.

Implication. Region-heterogeneous arrays are demanded and proven in the scientific, HPC, and graphics segments, and per-region precision is independently demanded in ML inference — but the two communities want different composition rules. A general format should therefore describe composition as a named rule over member tensors, rather than committing to a single geometry.

8. Gap Analysis: Required Capabilities

The survey supports a specific claim: **the transports are adequate and the descriptions are not**. This section states the capabilities a general tensor interchange format must provide, each traced to the evidence above. Together they form the design brief for **Hurray**, a format built around exactly these properties. The intent here is to state what is required, not to describe any particular realization.

8.1 Zero-copy sharing with stated alignment

Evidence: § 3.2 (buffer sizes); § 4.2.1 (Arrow specifies 64-byte alignment and benefits from it); § 4.2.2 (Flight loses alignment through gRPC and must copy); § 5.2.1 (numerical libraries adopt foreign memory routinely when its layout and alignment are known).

A consumer can use a producer’s buffer in place only if alignment, ownership, and lifetime are guaranteed in advance. Alignment left to convention becomes a copy at the boundary, and a copy at the boundary at gigabyte scale defeats the purpose of the exchange. A format must therefore state a minimum alignment normatively — sufficient for vector units, and stricter where accelerator and IPC paths require page alignment — and define lifetime transfer explicitly, as DLPack’s destructor callback does.

8.2 Streaming with self-delimiting framing

Evidence: § 4.2.2 (Flight’s message shape is right); § 4.3.1 and § 4.3.2 (file formats are not streamable in the general case); § 6 (transfers are continuous and per request, not whole-artifact).

A reader must be able to begin work without buffering the entire input, and a writer must be able to emit tensors one at a time without buffering the entire output. This has a concrete structural consequence: each tensor’s description must precede its data, the stream must be self-delimiting, and back-references or trailing indexes must be absent from the streaming form — because a consumer cannot seek to a footer in a stream that has not finished arriving.

8.3 Self-description

Evidence: § 6.7 (every production system agrees the description out of band); § 6.3 (fixing the format at registration couples endpoints); § 6.6 (compressed payloads are unreadable without their codec); § 4.3.4 (scaling by attribute convention is interpreted inconsistently).

The description must travel with the bytes: shape, element type, layout, quantization, device placement, and — where the tensor is one piece of a larger logical array — its position and extent within that array. This is the capability whose absence produces every symptom in § 6.7, and the one that turns a stored buffer from an engine-private artifact into durable data.

8.4 Layout negotiation

Evidence: § 3.1 (no layout is universally optimal); § 5.2.3 (serious numerical libraries maintain several layouts simultaneously); § 5.3.1 (compilers rewrite into packed and tiled forms); § 6.4 (mismatched layouts are reconciled by hand-written conversion).

Two capabilities are needed, and they are distinct. First, a **layout vocabulary** rich enough to name what producers actually hold: strided, tiled or blocked, space-filling-curve orders, the sparse families, indirect page-table layouts, and composition of heterogeneous regions — plus an extension mechanism for hardware-specific packed forms that cannot be enumerated in advance. Second, **negotiation**: each side declares what it can consume, so that conversion happens once, on the side best placed to perform it, rather than unconditionally at the receiver. A vocabulary without negotiation merely relocates the problem.

8.5 Device and memory-placement description

Evidence: § 4.4.1 (RDMA registration is memory-region-specific); § 5.4.1 (unified memory has no host/device duality); § 6.4 (production caches are tiered across accelerator memory, host memory, and storage).

A descriptor must say where a buffer lives, in a model admitting host memory, discrete accelerator memory, unified memory, and pinned or registered regions. The unified-memory finding of § 5.4.1 constrains the design: a model assuming every buffer belongs to exactly one device misdescribes an entire hardware class, so device affinity should be expressible as a property of access rather than assumed as a property of storage.

8.6 First-class quantization metadata

Evidence: § 4.3.2 (GGUF proves the value, and shows the cost of informal specification); § 5.1.2 and § 5.1.3 (quantization parameters travel separately from the tensor); § 5.4.1 (multiple packing conventions exist and are not interconvertible); § 7.2 (per-region precision is independently demanded).

Quantization is the normal case in inference rather than an exception, so scales, zero points, block or group sizes, and a scheme identifier must be part of the tensor description. Two properties matter beyond mere presence. The packing order for sub-byte elements must be specified bit-exactly, since reasonable alternatives exist. And the scheme set must be normative and versioned rather than defined by a reference implementation — the precise gap between GGUF’s practical excellence and its portability.

8.7 Language-agnostic ABI

Evidence: § 4.2.1 (Arrow’s C data interface works); § 4.1.1 (DLPack’s reach comes from being ABI-level); § 5.2.1 and § 5.2.2 (C++ numerical libraries adopt foreign buffers readily); § 4.3.1 (file formats without an ABI stop at the file boundary).

The boundary must be expressible in a stable C ABI, with no idioms specific to the implementation language leaking into it. That is what allows a format to be implemented independently in several languages rather than becoming one library with bindings.

8.8 Summary

#	Capability	Absorbed today by	Principal evidence
1	Zero-copy with stated alignment	Defensive copies at every boundary	§ 3.2, § 4.2.2
2	Streaming, self-delimiting framing	Whole-artifact file loads; buffering	§ 4.2.2, § 4.3, § 6
3	Self-description travelling with the bytes	Startup-time out-of-band agreement	§ 6.3, § 6.7
4	Layout vocabulary and negotiation	Repacking; hand-written conversion modules	§ 5.2.3, § 5.3.1, § 6.4
5	Device and memory-placement description	Engine-private placement configuration	§ 5.4.1, § 6.4
6	First-class quantization metadata	Config files and framework-private wrappers	§ 4.3.2, § 5.1.2
7	Language-agnostic ABI	Per-language bindings around a single library	§ 4.1.1, § 4.2.1

Table 5: Required capabilities and their evidence

No surveyed solution provides more than three of the seven. DLPack provides 1 and 7 for the in-process case. Arrow provides 1, 2, and 7 for a tabular data model. GGUF provides 3 and 6 for one runtime, on disk. The RDMA transports provide none, by design, and are the better for it: they are a data plane, and what is missing is the metadata plane above them.

9. Conclusion and Open Questions

9.1 Conclusion

Three findings summarize the survey.

Interchange solutions supply the foundations but not the vocabulary. DLPack and Arrow Flight each got something essential right — a zero-copy in-process ABI, and a streaming RPC shape in which the description precedes the data — and stopped short of the layout, quantization, and device description that inference workloads need. The RDMA transports move bytes at hardware speed and describe nothing, deliberately.

Compute frameworks pay for the shortfall, individually and repeatedly. PyTorch keeps quantization parameters outside the tensor; JAX cannot express sharding across a handoff; vLLM fixes its cache format at startup and ships opaque blocks; TensorRT-LLM hand-writes a layout converter. None of these is a defect in those systems. Each is the workaround an inexpressive interchange layer forces on a competent engineering team, which is why the same workaround keeps reappearing in unrelated code.

The missing artifact is a description, not a transport. The bytes already move fast enough. What does not exist is a portable statement of what they are — carried with them across in-process, IPC, file, and RDMA paths alike, and expressive enough to name a tiled tensor, a paged cache, a block-quantized weight matrix, or a heterogeneous composition. The seven capabilities of § 8 outline such a format, and constitute the design brief for **Hurray**.

9.2 Open questions

Designing to that brief raises questions this survey can pose but not settle.

1. **How large should the layout vocabulary be?** Every named layout is a conformance burden on every implementation, while every omission forces a copy. The evidence supports a small mandatory core plus an extension mechanism for hardware-specific forms — but where the line falls, and whether extension layouts can be negotiated meaningfully between parties that do not understand them, is unresolved.
2. **What should negotiation actually negotiate?** Declaring supported layouts is necessary but possibly insufficient: a producer able to emit two forms must choose, and the better choice depends on the relative cost of conversion at each end — information neither side currently has any way to express.
3. **Does device affinity belong to a buffer or to an access?** Section 5.4.1 shows that the conventional per-buffer model misdescribes unified memory. A per-access model is more faithful but complicates every descriptor that does not need it.
4. **How should quantization schemes evolve without proliferating?** New schemes appear faster than any specification revises. Parameterizing a small number of families is more durable than enumerating schemes, but only if the parameterization anticipates the axes along which new schemes actually vary.
5. **How much composition belongs in an interchange format?** Section 7 shows real demand for heterogeneous composition and two incompatible composition rules. Supporting both is expressive; supporting neither is simple; supporting one silently changes the meaning of overlapping indices.
6. **What is the right relationship to storage formats?** Section 4.3.3 suggests that pairing a compressed storage format with an uncompressed runtime format is a proven pattern. Whether one format should span both roles, or two should interoperate cleanly, determines whether compression belongs in the design at all.

Appendix A — Glossary

Affine quantization. Storing a value as a low-precision integer with a scale and, optionally, a zero point, so that the original value is approximated by $\text{scale} \times (\text{quantized} - \text{zero_point})$. Parameters may apply per tensor, per channel, or per block of consecutive elements.

Compute framework/library. A system that performs computation on tensors — kernels, graph execution, autotuning, request scheduling. The client of an interchange solution.

Data interchange solution. A format, protocol, or library whose purpose is to move or store tensor data. It defines bytes, descriptors, or transports, not arithmetic.

Disaggregated inference. Running the prefill and decode phases of language-model inference on separate accelerators or nodes, so that each can be scaled and tuned independently — which requires transferring the KV cache between them.

Group / block size. The number of consecutive elements sharing one set of quantization parameters. Smaller groups cost more metadata and lose less accuracy.

KV cache. The stored attention keys and values from previous tokens, which let a language model generate the next token without recomputing attention over the entire prompt. Logical shape `[layers, 2, heads, seq_len, head_dim]`.

Paged attention. A KV cache organization in which a flat pool of fixed-size blocks, plus a per-sequence block table mapping logical positions to physical blocks, replaces one contiguous buffer per sequence. It eliminates fragmentation and makes prefix sharing a matter of pointing two sequences at the same block.

Prefill / decode. The two phases of autoregressive generation. Prefill processes the whole prompt at once and is compute-bound; decode emits one token at a time and is memory-bandwidth-bound.

RDMA. Remote Direct Memory Access — one machine’s network interface reading or writing another machine’s registered memory without involving the remote processor. It requires exchanging a remote key and address beforehand. **Accelerator-direct RDMA** extends this to accelerator memory, with no staging copy through host memory.

SIMD. Single Instruction, Multiple Data — processor instructions applying one operation to several values simultaneously. SIMD kernels require operands to be contiguous and aligned, which is why alignment is a format concern rather than an implementation detail.

Strided layout. A layout described by one step size per dimension. It expresses row-major and column-major order, transposes, and slices; it cannot express tiling or packing.

Tensor parallelism / pipeline parallelism. Splitting a model across devices by dividing individual weight matrices (tensor parallelism) or by assigning whole layers to different devices (pipeline parallelism). When two disaggregated stages use different degrees, their caches are laid out differently — the mismatch described in § 6.4.

Tiled (blocked) layout. A layout storing small rectangular blocks contiguously so that each block fits in cache. Standard in high-performance dense linear algebra.

Zero-copy. Handing a consumer the producer’s existing memory rather than a duplicate. It requires prior agreement on alignment, ownership, and lifetime, which makes it a protocol property rather than an implementation technique.

Appendix B — Representative Tensor Shapes

Which layouts and alignment rules matter follows from the sizes that actually occur. Figures below are for a 70-billion-parameter transformer in 16-bit floating point.

Weights

Tensor	Shape	Size
Token embedding	[128256, 8192]	~2 GB
Attention query/key/value projection	[8192, 8192] each	~128 MB
Feed-forward gate/up projection	[8192, 28672]	~448 MB

Activations (dynamic)

Tensor	Typical shape
Input token embeddings	[B=32, S=2048, D=8192]
Attention scores	[B=32, H=64, S=2048, S=2048]
KV cache (all layers)	[2, 80, 32, 64, 2048, 128]

Vision — convolutional feature maps

Layer	Shape
Input batch	[32, 3, 224, 224]
Early convolution output	[32, 64, 112, 112]
Late convolution output	[32, 2048, 7, 7]

Thresholds

- Fits in mid-level cache: [64, 64] — 8 KB
- Fits in accelerator on-chip memory: [2048, 2048] — 8 MB
- A single weight matrix: [8192, 28672] — 448 MB
- Pathological (quadratic attention): [32, 64, 32768, 32768] — terabyte scale

Appendix C — Memory Layouts in Production

Layout	Best for	Notes
Row-major (C order)	Left matrix-multiplication operand, activations, attention scores	Default in most frameworks
Column-major (Fortran order)	Right matrix-multiplication operand, BLAS conventions	Default in classical numerical libraries
Tiled / blocked	High-intensity matrix multiplication, convolutions	Cache-optimal; requires repacking
Panel-packed	Innermost matrix-multiplication kernels	Ephemeral; repacking cost amortized over the kernel
Channel-last	Inference convolutions	Channel-contiguous, vector-unit friendly
Channel-first	Training convolutions	Historical default of early accelerator libraries
Paged	KV cache in autoregressive serving	Supports variable-length sequences and prefix sharing
Compressed sparse rows / blocks	Sparse weight matrices	Post-pruning inference
Structured 2:4 sparsity	Sparse matrix units	Requires a metadata mask

Layout	Best for	Notes
Sub-byte packed	Quantized weights	Block structure with interleaved scales

References

- [1] DLPack: Open In-Memory Tensor Structure. <https://github.com/dmlc/dlpack>
- [2] Apache Arrow: A cross-language development platform for in-memory data. <https://arrow.apache.org>
- [3] Apache Arrow Flight: A framework for high-performance data services. <https://arrow.apache.org/docs/format/Flight.html>
- [4] OPeNDAP: Open-source Project for a Network Data Access Protocol (DAP2/DAP4). <https://www.opendap.org>
- [5] SafeTensors: Safe serialization for tensors. Hugging Face. <https://github.com/huggingface/safetensors>
- [6] GGUF: GPT-Generated Unified Format. <https://github.com/ggerganov/ggml/blob/master/docs/gguf.md>
- [7] Zarr: Chunked, compressed, N-dimensional arrays. <https://zarr.dev>
- [8] NetCDF: Network Common Data Form. Unidata / UCAR. <https://www.unidata.ucar.edu/software/netcdf/>
- [9] NIXL: NVIDIA Inference Xfer Library. <https://github.com/ai-dynamo/nixl>
- [10] NCCL: NVIDIA Collective Communications Library. <https://developer.nvidia.com/nccl>
- [11] P. Shamis et al., “UCX: An Open Source Framework for HPC Network APIs and Beyond,” *IEEE Symposium on High-Performance Interconnects (HOTI)*, 2015. <https://openucx.org>
- [12] C. R. Harris et al., “Array programming with NumPy,” *Nature* 585, 357–362, 2020. <https://numpy.org>
- [13] A. Paszke et al., “PyTorch: An Imperative Style, High-Performance Deep Learning Library,” *NeurIPS*, 2019. <https://arxiv.org/abs/1912.01703>
- [14] M. Abadi et al., “TensorFlow: A System for Large-Scale Machine Learning,” *OSDI*, 2016. <https://arxiv.org/abs/1605.08695>
- [15] JAX: Composable transformations of Python and NumPy programs. <https://docs.jax.dev>
- [16] Eigen: A C++ template library for linear algebra. <https://eigen.tuxfamily.org>
- [17] xtensor: Multi-dimensional arrays with broadcasting and lazy computing. <https://xtensor.readthedocs.io>
- [18] PLASMA: Parallel Linear Algebra Software for Multicore Architectures. <https://icl.utk.edu/plasma/>
- [19] M. Gates et al., “SLATE: Software for Linear Algebra Targeting Exascale,” SLATE Working Notes, Innovative Computing Laboratory. <https://icl.utk.edu/slate/>
- [20] T. Chen et al., “TVM: An Automated End-to-End Optimizing Compiler for Deep Learning,” *OSDI*, 2018. <https://arxiv.org/abs/1802.04799>
- [21] MLC-LLM: Universal LLM deployment engine with machine learning compilation. <https://llm.mlc.ai>

- [22] MLX: An array framework for Apple silicon. <https://ml-explore.github.io/mlx/>
- [23] W. Kwon et al., “Efficient Memory Management for Large Language Model Serving with PagedAttention,” *SOSP*, 2023. <https://arxiv.org/abs/2309.06180>
- [24] NVIDIA Dynamo: A low-latency distributed inference framework. <https://developer.nvidia.com/blog/introducing-nvidia-dynamo-a-low-latency-distributed-inference-framework-for-scaling-reasoning-ai-models/>
- [25] Disaggregated Serving in TensorRT-LLM. NVIDIA. https://nvidia.github.io/TensorRT-LLM/blogs/tech_blog/blog5_Disaggregated_Serving_in_TensorRT-LLM.html
- [26] Y. Zhong et al., “DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving,” *OSDI*, 2024. <https://arxiv.org/abs/2401.09670>
- [27] R. Qin et al., “Mooncake: A KVCache-centric Architecture for Serving LLM Chatbot,” *USENIX FAST*, 2025. <https://arxiv.org/abs/2407.00079>
- [28] vLLM: Disaggregated prefilling and KV cache transfer connectors. https://docs.vllm.ai/en/stable/features/disagg_prefill/
- [29] llm-d: Kubernetes-native distributed inference. <https://llm-d.ai/docs/guide/Installation/pd-disaggregation>
- [30] LMCache: An open-source KV cache layer for LLM serving. <https://arxiv.org/html/2510.09665v2>
- [31] Y. Liu et al., “CacheGen: KV Cache Compression and Streaming for Fast Large Language Model Serving,” *ACM SIGCOMM*, 2024. <https://arxiv.org/abs/2310.07240>
- [32] J. Yao et al., “CacheBlend: Fast Large Language Model Serving for RAG with Cached Knowledge Fusion,” *EuroSys*, 2025. <https://arxiv.org/abs/2405.16444>
- [33] W. Zhang et al., “AMReX: A Framework for Block-Structured Adaptive Mesh Refinement,” *Journal of Open Source Software*, 2019. <https://amrex-codes.github.io>
- [34] P. Colella et al., “Chombo Software Package for AMR Applications — Design Document,” Lawrence Berkeley National Laboratory.
- [35] K. Museth, “VDB: High-Resolution Sparse Volumes with Dynamic Topology,” *ACM Transactions on Graphics*, 2013; and “NanoVDB: A GPU-Friendly and Portable VDB Data Structure,” 2021. <https://www.openvdb.org>
- [36] HDF5 Virtual Datasets. The HDF Group. https://docs.hdfgroup.org/hdf5/develop/_v_d_s.html
- [37] S. Papadopoulos et al., “The TileDB Array Data Storage Manager,” *VLDB*, 2016. <https://tiledb.com>
- [38] Zarr Enhancement Proposal: variable-size chunking. <https://zarr.dev/zeps/>
- [39] A. J. C. Bik et al., “Compiler Support for Sparse Tensor Computations in MLIR,” *ACM Transactions on Architecture and Code Optimization*, 2022. <https://mlir.llvm.org/docs/Dialects/SparseTensorOps/>
- [40] T. Dettmers et al., “SpQR: A Sparse-Quantized Representation for Near-Lossless LLM Weight Compression,” 2023. <https://arxiv.org/abs/2306.03078>
- [41] C. Hooper et al., “KVQuant: Towards 10 Million Context Length LLM Inference with KV Cache Quantization,” 2024. <https://arxiv.org/abs/2401.18079>
- [42] Z. Liu et al., “KIVI: A Tuning-Free Asymmetric 2bit Quantization for KV Cache,” 2024. <https://arxiv.org/abs/2402.02750>
- [43] M. Zaheer et al., “Big Bird: Transformers for Longer Sequences,” *NeurIPS*, 2020. <https://arxiv.org/abs/2007.14062>

- [44] J. Nystad et al., “Adaptive Scalable Texture Compression,” *High Performance Graphics*, 2012.
- [45] Open Compute Project, “OCP Microscaling Formats (MX) Specification,” v1.0, 2023. <https://www.opencompute.org/documents/ocp-microscaling-formats-mx-v1-0-spec-final-pdf>
- [46] NVIDIA GPUDirect RDMA documentation. <https://docs.nvidia.com/cuda/gpudirect-rdma/>
-

Survey compiled April 2026; disaggregated-inference case study added June 2026; region-heterogeneous structures added July 2026; restructured as a state-of-the-art review September 2026. Systems are described as of their most recent publicly documented behaviour at those dates.